Quantitative Colour Pattern Analysis (QCPA)

Example post-batch processing and analysis in R

Contents

1 Setup **2**

2 Create the foundational dataset identifying unique animal IDs and regions of interest for multiple viewing distances **2**

2.1 Iterate over folders to extract species, animals and viewing distances 2

2.2 Inspect data 3

2.3 Inspect regions of interest for inconsistent labelling (additional step) 6

2.4 Add regions of interest to animal information dataset 6

2.5 Merge the list of ROIs names with the animal ID dataframe 8

3 Create a function to import a .csv list from VCA, BSA, and CAA data, with options for particle analysis **9**

3.1 Simplified function example to read in image analysis files 9

3.2 Dynamic function to read in various image analysis files 11

4 Get a .csv list from LEIA output **15**

5 Get a .csv list from GabRat output **16**

6 Combine data sets and data preparation **17**

6.1 Merging data sets 17

6.2 Data cleaning 20

7 Export files **30**

8 Fast import for large data sets using data.table (optional - advanced) **31**

References **33**

1 Setup

The following R script uses the trichromatic example without UV (sea slugs seen by a triggerfish, *Rhinecanthus aculeatus*), but can similarly be applied to the tetrachromatic example with UV (spiders seen by a bluetit, *Parus major*). To make the worked examples readily accessible to users, we have prioritised using base R functions. As we will be working with primarily character-type data (i.e., species names, region of interest descriptors), we will introduce the user to many base functions that are for list-type data structures (e.g., `lapply()`). For clarity, we recommend that you, as the user, need to be comfortable with the different data types (e.g., numerical, character, factors) and data structures such as lists (collection of mixed data types) and data frames (special type of list). We also provide different methods to extract data, ideally introducing different coding methods and providing ways to increase the efficiency of data wrangling.

For the following R script to run, please ensure that you have implemented the QPCA batch script methods. In the following folder

`example_data/eg1_trichromat_nouv/test_data`

there should be two subfolders for the two species, *Aphelodoris varia* and *Aplysia sp*. Within these subdirectories, please confirm that data has been generated. As you proceed through the next few steps, we provide code that checks the data completeness. If you are uncertain that you uniformly named files, we have additionally provided script to check region of interest names (see section 2.3).

2 Create the foundational dataset identifying unique animal IDs and regions of interest for multiple viewing distances

2.1 Iterate over folders to extract species, animals and viewing distances

First, we will specify the location of the data files with respect to where your working directory has been saved. Here, we have assumed that the R script was saved in the parent folder of `example_data`. The next step will be to extract the names of the two focal species using the folder names within the `test_data` folder.

```
# Construct the global data location path. This assumes the R script is in the
# project root
data_location <- file.path(getwd(), "example_data/eg1_trichromat_nouv/test_data")

# List the species folders in the data location
# These could also be locations depending on your data set
species_folders <- list.files(path = data_location)

# Print the list of species folders and log file with QCPA processing details
print(species_folders)
```

```
## [1] "aphelodoris_varia"  "aplysia_sp"         "Batch_QCPA_Log.txt"
```

```
# Filter species folders based on the "Ap" pattern in their names
# This removes the log file from the vector
species <- grep("ap", species_folders, value = TRUE)

# Note: For multiple different species patterns, can search using the or
# condition '|'. e.g., species <- species[grepl("A|B", species)],
# returns species with pattern 'A' or 'B'
```

```r
# Print the filtered list of species folders
print(species)
```

```
## [1] "aphelodoris_varia" "aplysia_sp"
```

Here, we will demonstrate extracting folder information names using a for loop. We first create an empty data frame called `animal_info`. Then, for each ith species (here $i = 1$ or 2), we create a path into the species folder and record in the object `Indviduals_list` all the unique individual animal folder names using the `list.files` function. We then establish a second, nested loop, which will then iterate over each individual (`ind`) in `Indviduals_list` to extract all unique viewing distance folder names using `list.files`. To distinguish the viewing distance folders from other files saved within the directory, use the argument `pattern = "cm"` in `list.files`. Finally, we combine the unique species name, individual name and viewing distances into unique columns and add as a new row in `animal_info`.

```r
# Initialize an empty data frame
animal_info <- data.frame()

# Iterate over the species folders
for (species_name in species) {

  species_path <- file.path(data_location, species_name)
  individual_list <- list.files(path = species_path)

  # Iterate over the individuals in each species folder
  for (individual_name in individual_list) {

    individual_path <- file.path(species_path, individual_name)
    distances <- list.files(path = individual_path, pattern = "cm")

    # Add new rows to the animal_info data frame
    new_rows <- expand.grid(
      Species = species_name,
      Ind = individual_name,
      Dist = distances,
      stringsAsFactors = FALSE
    )
    animal_info <- rbind(animal_info, new_rows)
  }
}

# Rename the column names in animal_info
colnames(animal_info) <- c("Species", "Ind", "Dist")
```

2.2 Inspect data

Check that the data has correctly been imported and reflects your experimental records. We converted the character data into factors, which is the appropriate format for running statistical tests in downstream analyses. We use the `lapply()` function to apply the function `factor()` to each column in our data frame. We can again use the `lapply()` function to apply the function `nlevels()` to our newly converted data frame, which will report the number of unique elements within each column of the data frame `animal_info`.

```r
# Check data
head(animal_info)
```

```
##             Species                            Ind Dist
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm
## 3 aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 30cm
## 4 aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 50cm
## 5        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 30cm
## 6        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 50cm
```

```r
# Convert character columns to factors
animal_info[] <- lapply(animal_info, as.factor)

# Display structure of the data
str(animal_info)
```

```
## 'data.frame':    6 obs. of  3 variables:
##  $ Species: Factor w/ 2 levels "aphelodoris_varia",..: 1 1 1 1 2 2
##  $ Ind    : Factor w/ 3 levels "3_AphelodorisVaria_10_NF_D_RAW",..: 1 1 2 2 3 3
##  $ Dist   : Factor w/ 2 levels "30cm","50cm": 1 2 1 2 1 2
##  - attr(*, "out.attrs")=List of 2
##   ..$ dim     : Named int [1:3] 1 1 2
##   .. ..- attr(*, "names")= chr [1:3] "Species" "Ind" "Dist"
##   ..$ dimnames:List of 3
##   .. ..$ Species: chr "Species=aphelodoris_varia"
##   .. ..$ Ind    : chr "Ind=3_AphelodorisVaria_10_NF_D_RAW"
##   .. ..$ Dist   : chr [1:2] "Dist=30cm" "Dist=50cm"
```

```r
# Check the number of unique species, individuals and viewing distances
sapply(animal_info, nlevels)
```

```
## Species     Ind    Dist
##       2       3       2
```

```r
# Aggregate data to count individuals by species and distance
aggregate_counts <- aggregate(Ind ~ Species + Dist, data = animal_info, FUN = length)

# Plot bar chart of individual counts by species and distance
barplot(Ind ~ Dist + Species,
  beside = TRUE,
  ylab = "Count",
  legend.text = TRUE,
  data = aggregate_counts
)
```

In this example, there are only three individuals. However, with a larger data set with missing data, this becomes an important step to determine a balanced experimental design.

Figure 1: Figure 1. Counts of individuals within species for each viewing distance.

2.3 Inspect regions of interest for inconsistent labelling (additional step)

The number of data files will depend on whether animal, background and/or animal+background options were selected; this will make up the prefix of the file name followed by an underscore and image analysis file type (e.g., '*_Summary Results.csv*'). For the following R code to work, regions of interest must be uniformly named and capitalised for all individuals analysed. Below is a method to ensure that these regions of interest (ROIs) have been correctly named by reporting all unique names across your data by examining all unique `.tif` file names. If you are confident that the naming was consistent, you may skip this step.

```
# Generate individual animal sub-directories paths
animal_sub_dirs <- apply(animal_info, 1, function(x) paste(x, collapse = "/"))

# Prepend the global file directories to the sub-directory paths
full_animal_dirs <- file.path(data_location, animal_sub_dirs)

# Identify the unique regions of interest. If there are
# more than expected - look for naming inconsistencies.
tif_files <- list.files(full_animal_dirs, pattern = ".tif")
roi_names <- unique(unlist(strsplit(tif_files, "_"))[c(TRUE, FALSE)])
roi_names
```

```
## [1] "animal"            "animal+background" "background"
```

How the final lines works

The final lines achieve four things:

1. `list.files()` searches all the locations specified in the `full_animal_dirs` vector for files with the `.tif` extension and puts these file names into a new character vector.
2. Knowing that file names are composed of two bits of information separated by an underscore, `strsplit()` cuts the filenames into two elements at the underscore and outputs a list containing these two new elements for each unique file.
3. `unlist()` puts all elements into a vector. As the region of interest are the prefix, it will be first element in the pair, which we can subset using: `[ c(TRUE,FALSE)]`.
4. Finally, the `unique()` function looks across all names used for each individual and retains only unique names, omitting duplicates.

2.4 Add regions of interest to animal information dataset

To proceed with the next step, the regions of interest (ROIs) must be uniformly named across all individual specimens (see above if uncertain). Here, we introduce two methods to add ROIs. The first method is the quickest, and works if you are confident in your naming conventions.

However, we provide a second method that extracts your data's ROIs.

2.4.1 Simplified method to list ROIs

```
ROI <- c("animal", "animal+background", "background") # Add the names of ROIs
```

2.4.2 Extract ROIs from the data

Here, we will utilise the batch script naming convention of using the ROI as the prefix of the image analysis file name followed by an underscore and image analysis file type (e.g., '*_Summary Results.csv*'). See Table 1 below for all QCPA analyses and associated file names.

Table 1: Table 1. QCPA image analysis and associated output files. See van den Berg & Troscianko et al. (2020) for a detailed description.

Image analysis	File name
CAA, VCA, BSA	_Summary Results.csv
GabRat	_GabRat_Results.csv
LEIA	_Local Edge Intensity Analysis.csv
Particle analysis	_Cluster Particle Analysis Summary Results.csv
Particle analysis + cluster properties	_Cluster Results.csv
Particle analysis + sub-cluster properties	_Individual Particle Results.csv
Particle analysis + transition matrix + cluster properties	_ROI Cluster Results.csv

We can then extract any particular image analysis `.csv` data files names using the `list.files()` function specifying the file name as the pattern to search for. In the following code, we have used '_Summary Results.csv' as our string pattern. Once we have all the file names with '_Summary Results.csv', we remove this phrase using `gsub()`.

```
# Here, we create a character vector determining whether animal,
# background and/or animal+background options were used
# Using the first row of animal_info to specify the location to examine
# the number and types of _Summary Results.csv files

# Use the first row of animal_info to specify the location to examine
first_animal_path <- file.path(data_location, paste(unlist(animal_info[1, ]), collapse = "/"))

# Find all the ROI output files for '_Summary Results.csv'
roi_files <- list.files(first_animal_path, pattern = "_Summary Results.csv")

# Extract ROI names by removing the "_Summary Results.csv" suffix
rois <- gsub("_Summary Results.csv", "", roi_files)
rois
```

```
## [1] "animal"            "animal+background" "background"
```

```
# Note: In the QCPA batch script, three ROIs were investigated.
```

2.5 Merge the list of ROIs names with the animal ID dataframe

```
# Expand each unique row of the animal_info data frame to included a unique ROI value
Animal_Info_ROI <- merge(animal_info, rois)

colnames(Animal_Info_ROI)[4] <- "ROI"  # Name column 4 to ROI

Animal_Info_ROI$ROI <- factor(Animal_Info_ROI$ROI)  # Convert to factor

head(Animal_Info_ROI)  # Check data
```

```
##             Species                              Ind Dist    ROI
```

```
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal
## 3 aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 30cm animal
## 4 aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 50cm animal
## 5        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 30cm animal
## 6        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 50cm animal
```

```r
str(Animal_Info_ROI)  # Check that ROI is a factor
```

```
## 'data.frame':    18 obs. of  4 variables:
##  $ Species: Factor w/ 2 levels "aphelodoris_varia",..: 1 1 1 1 2 2 1 1 1 1 ...
##  $ Ind    : Factor w/ 3 levels "3_AphelodorisVaria_10_NF_D_RAW",..: 1 1 2 2 3 3 1 1 2 2 ...
##  $ Dist   : Factor w/ 2 levels "30cm","50cm": 1 2 1 2 1 2 1 2 1 2 ...
##  $ ROI    : Factor w/ 3 levels "animal","animal+background",..: 1 1 1 1 1 1 2 2 2 2 ...
```

3 Create a function to import a .csv list from VCA, BSA, and CAA data, with options for particle analysis

As before, you could use a series of nested loops to extract the data in the csv files (e.g., section 2.1). However, here we introduce how to define a function to determine the data location using the data listed in the *Animal_Info_ROI* data frame and import the .csv file. By creating a function, we can then make our file importing dynamic, providing options to import different image analysis files. And, by using information called directly from the *Animal_Info_ROI* data frame, we ensure the data extracted is correct and reduces errors that would have potentially occurred if we did a broad import. First, we present a simple example of a function to import the VCA,BSA,CAA data files, and then, secondly, we expand on our function to demonstrate how to define and import multiple types of image analysis data files.

3.1 Simplified function example to read in image analysis files

In the following function, we walk through the steps for a function that only imports VCA, BSA, CAA image analysis data but will lay the foundations for when we create the function that can input any of the image analyses.

There are three parts:

1. The first section will specify what type of image analysis we are interested in
2. The second section extracts the data using `Animal_Info_ROI` data frame and stores the data as a list in the object `Sum_Res_list`
3. In the third section, we use the `do.call()` function to extract all the list elements in `Sum_Res_list` and combine them into a data frame using `rbind()`. From Table 1, the file suffix for this analysis is '*_Summary Results.csv*'.

```r
# Part A:
# Here, we specify what image analysis so that we can use the correct file extension
exnfile <- "_Summary Results.csv"

Ani_ID_dat <- Animal_Info_ROI # Assign our animal ID data frame

path <- data_location # This is the global location set in section 2.1
```

```r
# Part B:
# Once the file extension condition is met, we use lapply to extract the data,
# using a nested function within lapply
Sum_Res_list <- lapply(
  1:nrow(Ani_ID_dat), # for each row in Ani_ID_dat...
  function(x) { # start of our nested function

    # Extract the individual row information
    tmpLoc <- paste(unlist(Ani_ID_dat[x, ]), collapse = "/")

    # Create a directory string
    tmpLoc <- paste0(paste(path, tmpLoc, sep = "/"), exnfile)

    # Read in the data using the newly specified location
    data <- read.csv(tmpLoc)

    # Add the specific animal information to the data
    data <- merge(Ani_ID_dat[x, ], data)

    data # Returns data
  }
)

# Part C:
# We then collapse the Sum_Res_list list into a data frame by
# calling rbind in the function do.call
Sum_Res_df <- do.call(rbind, Sum_Res_list)

# Check the imported data
dim(Sum_Res_df) # 18 rows by 133 columns
```

```
## [1]  18 133
```

```r
head(Sum_Res_df)[, 1:12] # check the first six rows and first twelve columns
```

```
##             Species                            Ind Dist    ROI X
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal 1
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal 1
## 3 aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 30cm animal 1
## 4 aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 50cm animal 1
## 5        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 30cm animal 1
## 6        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 50cm animal 1
##                                                                          Image   CAA.Sc    C
## 1 3_AphelodorisVaria_10_NF_D_RAW.ORF_Acuity3_Dist300_0_WeberFiltered_itr5_rad5_fall3_0 1.000000 1.00
## 2 3_AphelodorisVaria_10_NF_D_RAW.ORF_Acuity3_Dist500_0_WeberFiltered_itr5_rad5_fall3_0 1.000000 1.00
## 3 3_AphelodorisVaria_11_NF_D_RAW.ORF_Acuity3_Dist300_0_WeberFiltered_itr5_rad5_fall3_0 3.407769 0.85
## 4 3_AphelodorisVaria_11_NF_D_RAW.ORF_Acuity3_Dist500_0_WeberFiltered_itr5_rad5_fall3_0 3.321784 0.83
## 5    4_AplysiaParvula_1_NL_N_RAW.ORF_Acuity3_Dist300_0_WeberFiltered_itr5_rad5_fall3_0 2.095732 0.69
## 6    4_AplysiaParvula_1_NL_N_RAW.ORF_Acuity3_Dist500_0_WeberFiltered_itr5_rad5_fall3_0 1.020302 0.51
##       CAA.Qc
## 1        NaN
## 2        NaN
```

```
## 3 0.93318605
## 4 0.92335109
## 5 0.73432714
## 6 0.08112609
```

```
# Finally, to prevent any errors we remove the objects Ani_ID_dat, exnfile, path
# from the global environment because we'll be recycling them in the second function
rm(exnfile)
rm(Ani_ID_dat)
rm(path)
```

3.2 Dynamic function to read in various image analysis files

To dynamically import data, we are going to create the function `read_qcpa()`, which will take an animal ID data frame (`Ani_ID_dat`), the specific image analysis that we are interested in (`filetype`), and a folder directory (where we set the default `path` to the `data_location` defined in section 1.2). For Part A of the function, we can conditionally set what image analysis we are interested in and set the `exnfile` object to the correct file extension. We have listed each image analysis and the argument name to be specified as the filetype in Table 2. As the last conditional (at the `else` level), we have used the `stop` function which will occur if `filetype` has not been specified, and will report an error. Part B is the same as in section 3.1 above however it is worth noting that we used `merge` here instead of `rbind()` (section 2.1) because this allows our function to handle data in both long and wide format. Then, in Part C, we use the `return` function to output our newly imported data.

Table 2: Table 2. QCPA image analysis and the different filetype arguments

Image analysis	Filetype argument
VCA,BSA,CAA	"VCA"
Cluster Particle Analysis	"PartAn"
Cluster Results	"Clust"
Individual Particle	"IndParticle"

```
read_qcpa <- function(Ani_ID_dat, filetype = "NA", path = data_location) {
  # Default location is assigned, but can be changed

  # Part A: Specify arguments for different image analyses
  file_extensions <- list(
    VCA = "_Summary Results.csv",
    PartAn = "_Cluster Particle Analysis Summary Results.csv",
    Clust = "_Cluster Results.csv",
    IndParticle = "_Individual Particle Results.csv"
  )

  if (!filetype %in% names(file_extensions)) {
    stop('Specify analysis output type: filetype = "VCA", "PartAn", "Clust", or "IndParticle"')
  }

  exnfile <- file_extensions[[filetype]]

  # Part B: Extract data using lapply
```

```r
  data_list <- lapply(seq_len(nrow(Ani_ID_dat)), function(x) {
    # Extract row information and create directory string
    dir_components <- unlist(Ani_ID_dat[x, ])
    tmpLoc <- file.path(path, paste(dir_components[1:3], collapse = "/"),
                        paste(dir_components[4], exnfile, sep = ""))

    # Check if file exists before trying to read it
    if (!file.exists(tmpLoc)) {
      stop("File does not exist: ", tmpLoc)
    }

    # Read data and add animal information
    data <- read.csv(tmpLoc)
    data <- merge(Ani_ID_dat[x, ], data)

    return(data)
  })

  # Part C: Combine list into a data frame
  data_df <- do.call(rbind, data_list)

  return(data_df)
}
```

To demonstrate how the stop warning works, we have implemented the `read_qcpa()` function here without specifying the image analysis file type.

```r
read_qcpa(Animal_Info_ROI) # We include only the animal id data set
```

```
## Error in read_qcpa(Animal_Info_ROI): Specify analysis output type: filetype = "VCA", "PartAn", "Clus
```

We can then import our data using our function and do some data cleaning by removing unnecessary columns.

```r
# VCA,BSA,CAA analysis:
# Read in and assign analysis
VCA_analysis <- read_qcpa(Animal_Info_ROI, filetype = "VCA")

# head(VCA_analysis) # Inspect data

# Subset data excluding the column Image and X
VCA_analysis <- VCA_analysis[, !colnames(VCA_analysis) %in% c("X", "Image")]

dim(VCA_analysis) # 18 rows, by 131 columns
```

```
## [1]  18 131
```

```r
VCA_analysis[1:2, ] # Observe the first two rows of data
```

```
##             Species                            Ind Dist    ROI CAA.Sc CAA.Jc CAA.St CAA.Jt CAA.Hc CA
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal      1      1    Inf    NaN      0
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal      1      1    Inf    NaN      0
```

```
##   CAA.C CAA.Sc.Hrz CAA.Sc.Vrt CAA.Jc.Hrz CAA.Jc.Vrt CAA.St.Hrz CAA.St.Vrt CAA.Jt.Hrz CAA.Jt.Vrt CAA.
## 1     0          1          1          1          1        Inf        Inf        NaN        NaN
## 2     0          1          1          1          1        Inf        Inf        NaN        NaN
##   CAA.Qc.Vrt CAA.Ht.Hrz CAA.Ht.Vrt CAA.Qt.Hrz CAA.Qt.Vrt CAA.Scpl.Hrz CAA.Scpl.Vrt CAA.Qcpl.Hrz CAA.
## 1        NaN          0          0        NaN        NaN          Inf          Inf          NaN
## 2        NaN          0          0        NaN        NaN          Inf          Inf          NaN
##   CAA.PT.Hrz CAA.PT.Vrt CAA.Asp     VCA.ML VCA.sL VCA.CVL  VCA.MDmax VCA.sDmax VCA.CVDmax VCA.MSsat
## 1        Inf        Inf     NaN 0.07415875    NaN     NaN 0.08914595       NaN        NaN  2.695919
## 2        Inf        Inf     NaN 0.07259081    NaN     NaN 0.08963545       NaN        NaN  2.723553
##   VCA.CVSL VCA.MS VCA.sS VCA.CVS VCA.ML.Hrz VCA.sL.Hrz VCA.CVL.Hrz VCA.ML.Vrt VCA.sL.Vrt VCA.CVL.Vrt
## 1      NaN    NaN    NaN     NaN 0.07415875        NaN         NaN 0.07415875        NaN         NaN
## 2      NaN    NaN    NaN     NaN 0.07259081        NaN         NaN 0.07259081        NaN         NaN
##   VCA.CVDmax.Hrz VCA.MDmax.Vrt VCA.sDmax.Vrt VCA.CVDmax.Vrt VCA.MSsat.Hrz VCA.sSsat.Hrz VCA.CVSsat.H
## 1            NaN    0.08914595           NaN            NaN      2.695919           NaN            N
## 2            NaN    0.08963545           NaN            NaN      2.723553           NaN            N
##   VCA.CVSsat.Vrt VCA.MSL.Hrz VCA.sSL.Hrz VCA.CVSL.Hrz VCA.MSL.Vrt VCA.sSL.Vrt VCA.CVSL.Vrt VCA.MS.Hr
## 1            NaN         NaN         NaN          NaN         NaN         NaN          NaN        Na
## 2            NaN         NaN         NaN          NaN         NaN         NaN          NaN        Na
##   VCA.sS.Vrt VCA.CVS.Vrt BSA.BML BSA.BsL BSA.BCVL BSA.BMDmax BSA.BsDmax BSA.BCVDmax BSA.BMSsat BSA.B
## 1        NaN         NaN       0       0      NaN          0          0         NaN          0
## 2        NaN         NaN       0       0      NaN          0          0         NaN          0
##   BSA.BCVSL BSA.BMS BSA.BsS BSA.BCVS BSA.BML.Hrz BSA.BsL.Hrz BSA.BCVL.Hrz BSA.BML.Vrt BSA.BsL.Vrt BSA
## 1       NaN       0       0      NaN           0           0          NaN           0           0
## 2       NaN       0       0      NaN           0           0          NaN           0           0
##   BSA.BsDmax.Hrz BSA.BCVDmax.Hrz BSA.BMDmax.Vrt BSA.BsDmax.Vrt BSA.BCVDmax.Vrt BSA.BMSsat.Hrz BSA.Bs
## 1              0             NaN              0              0             NaN              0
## 2              0             NaN              0              0             NaN              0
##   BSA.BMSsat.Vrt BSA.BsSsat.Vrt BSA.BCVSsat.Vrt BSA.BMSL.Hrz BSA.BsSL.Hrz BSA.BCVSL.Hrz BSA.BMSL.Vrt
## 1              0              0             NaN            0            0           NaN            0
## 2              0              0             NaN            0            0           NaN            0
##   BSA.BMS.Hrz BSA.BsS.Hrz BSA.BCVS.Hrz BSA.BMS.Vrt BSA.BsS.Vrt BSA.BCVS.Vrt
## 1           0           0          NaN           0           0          NaN
## 2           0           0          NaN           0           0          NaN
```

```r
# Cluster Particle Analysis:
# Read in and assign analysis
Particle_analysis <- read_qcpa(Animal_Info_ROI, filetype = "PartAn")

# head(Particle_analysis) # Inspect data

dim(Particle_analysis) # 52 rows, by 20 columns
```

```
## [1] 52 20
```

```r
colnames(Particle_analysis)[5]
```

```
## [1] "Cluster.ID"
```

```r
# Change column name to be consistent with Cluster analysis
colnames(Particle_analysis)[5] <- "ClusterID"

Particle_analysis[1:2, ] # Observe the first two rows of data
```

```
##             Species                            Ind Dist    ROI ClusterID Count Total.Area Average.Siz
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal         1     4       2953     738.250
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal         1     7       1084     154.857
##       Minor    Angle     Circ.  Solidity    Feret   FeretX   FeretY FeretAngle MinFeret
## 1 17.837978 79.13099 0.2135263 0.5150810 83.09385 19.25000 60.25000   79.74821 31.79423
## 2  6.734171 60.18813 0.4806954 0.7693886 27.63505 23.42857 30.42857  105.63732 10.86412
```

```r
# Cluster Results:
# Read in and assign analysis
Cluster_analysis <- read_qcpa(Animal_Info_ROI, filetype = "Clust")

# head(Cluster_Analysis) # Inspect data

# Subset data excluding the column Image and X
Cluster_analysis <- Cluster_analysis[, !colnames(Cluster_analysis) %in% c("X", "Image")]

dim(Cluster_analysis) # 52 rows, by 21 columns
```

```
## [1] 52 21
```

```r
Cluster_analysis[1:2, ] # Observe the first two rows of data
```

```
##             Species                            Ind Dist    ROI Pass ClusterID    lw_mean      lw_sd
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal    7         1 0.07855250 0.01874494 0
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal    6         1 0.07697873 0.01535109 0
##        sw_sd   dbl_mean     dbl_sd Area Coverage..pc.    RNL.X     RNL.Y RNL.Saturation Dmax.Chromati
## 1 0.01396244 0.07415875 0.01840701 9116           100 2.074853 -1.721327       2.695919        0.089
## 2 0.01060753 0.07259081 0.01485467 3375           100 2.117579 -1.712775       2.723553        0.089
```

```r
# Individual Particle Analysis:
# Read in and assign analysis
IndParticle_analysis <- read_qcpa(Animal_Info_ROI, filetype = "IndParticle")

# head(Cluster_Analysis) # Inspect data

# Subset data excluding the column X.1
IndParticle_analysis <- IndParticle_analysis[, colnames(IndParticle_analysis) != "X.1"]

dim(IndParticle_analysis) # 143 rows, by 23 columns
```

```
## [1] 143  23
```

```r
IndParticle_analysis[1:2, ] # Observe the first two rows of data
```

```
##             Species                            Ind Dist    ROI ClusterID_Particle.         X        Y
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal                 1_1  9.001362 10.64169
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal                 1_2 71.712538 16.22783
##      Major    Minor     Angle     Circ.    Feret FeretX FeretY FeretAngle MinFeret       AR     Round
## 1 29.94767 15.60318  47.74661 0.1786535 60.82763      0     42   43.66778 31.08705 1.919332 0.521014
## 2 45.31033 18.37768 109.97441 0.2730243 71.11259     50      0  115.84439 28.42651 2.465509 0.405595
```

4 Get a .csv list from LEIA output

Here, we implement a function similar to section 3.1. However, because QCPA batch script exports LEIA analysis into a subdirectory, we have to add two additional folders to our location, `LEIA` and the ROI as a folder. For the LEIA folder, as we iterate over each row from `Animal_Info_ROI`, we convert the row into a character vector and add 'LEIA' as an element. We include a backslash to treat the ROI as a folder before specifying the image analysis file extension, `_Local Edge Intensity Analysis.csv` (Table 1).

```
# Define a function to generate file paths, read data, and merge animal information
read_leia <- function(index, animal_info, base_location) {
  # Extract row as a character vector
  row_values <- as.character(unlist(animal_info[index, ]))

  # Construct the file path
  file_path <- file.path(
    base_location,
    row_values[1],
    row_values[2],
    row_values[3],
    "LEIA",
    row_values[4],
    "_Local Edge Intensity Analysis.csv"
  )

  # Read data
  data <- read.csv(file_path)

  # Add animal information to the data
  data <- merge(animal_info[index, ], data)

  return(data)
}

# Read LEIA data using lapply
LEIA_Res_list <- lapply(
  seq_len(nrow(Animal_Info_ROI)),
  function(x) read_leia(x, Animal_Info_ROI, data_location)
)

# Combine list into a data frame
LEIA_Res_analysis <- do.call(rbind, LEIA_Res_list)

# Subset data to exclude specific columns
LEIA_Res_analysis <- LEIA_Res_analysis[, !(colnames(LEIA_Res_analysis)
                                           %in% c("X", "Image"))]

# Check the resulting data
dim(LEIA_Res_analysis) # Print dimensions of the data
```

```
## [1] 18 37
```

```
head(LEIA_Res_analysis, 2) # Print the first two rows of data
```

```
##             Species                            Ind Dist    ROI  Col.mean    Col.sd  Col.CoV Col.skew
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal 0.1736449 0.2851837 1.642339 2.351301
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal 0.1663600 0.2695337 1.620183 2.286320
##    Lum.CoV Lum.skew Lum.kurtosis Col.mean.hrz Col.sd.hrz Col.CoV.hrz Col.skew.hrz Col.kurtosis.hrz Co
## 1 1.602808 2.255974     9.558368   0.08770301  0.1856711    2.117043     3.926095         22.61682
## 2 1.536144 2.060313     8.282279   0.08205933  0.1756506    2.140532     3.887552         22.67175
##   Col.skew.vrt Col.kurtosis.vrt Lum.mean.hrz Lum.sd.hrz Lum.CoV.hrz Lum.skew.hrz Lum.kurtosis.hrz Lum
## 1     3.913759         22.07321    0.4355562  0.8958863    2.056879     3.612730         20.01520
## 2     3.881844         22.90526    0.4406635  0.8761384    1.988225     3.493638         19.45551
##   Lum.skew.vrt Lum.kurtosis.vrt Transform Col.thresh Lum.thresh
## 1     3.852029         22.42973      none          0          0
## 2     3.690217         20.01364      none          0          0
```

5 Get a .csv list from GabRat output

For GabRat analyses, there are no specific ROIs, so we only need the first three columns from `Animal_Info_ROI`. Similar to LEIA (section 4, above), QCPA batch script also exports GabRat analysis into a subdirectory. We will again iterate over each row of `Animal_Info_ROI` and convert the row into a character vector, adding 'GabRat' as an element. From Table 1, the GabRat file extension is: `_GabRat_Results.csv`, where we place a backslash before the file name to specify that it is located within the GabRat folder.

For this work example, we are only interested in the GabRat values `dbl` (the luminance channel of the visual system considered in the analysis), so we will then extract the rows with only these values. We then extract the ROI information in a method similar to section 2.3.

```r
# Because we are not using ROI which adds three levels (rows),
# we trim data frame using unique()
# For each the 6 rows in Animal_Info_ROI

# Import GabRat results and process data

# Function to read and process GabRat results
read_gabrat <- function(data_location, animal_info) {
  gabrat_res_list <- lapply(
    1:nrow(unique(animal_info[, 1:3])),
    function(row_index) {
      # Convert row into a character vector
      tmp_loc <- as.character(unlist(animal_info[row_index, 1:3]))
      tmp_loc <- c(tmp_loc, "GabRat") # Add the new sub folder GabRat
      dir_path <- file.path(data_location, paste(tmp_loc, collapse = "/"), "_GabRat_Results.csv")

      # Read in the data using the newly specified location
      data <- read.csv(dir_path)

      # Add the specific animal information to the data
      data <- merge(animal_info[row_index, 1:3], data)
      data
    }
  )

  # Collapse the gabrat_res_list list into a data frame
  gabrat_res_analysis <- do.call(rbind, gabrat_res_list)
```

```r
  # Filter data for rows containing "_dbl" in ID
  gabrat_res_analysis <- gabrat_res_analysis[endsWith(gabrat_res_analysis$ID, "_dbl"), ]
  rownames(gabrat_res_analysis) <- NULL

  # Add ROI column by extracting substring after "WholeImage_"
  gabrat_res_analysis$ROI <- sapply(
    strsplit(gabrat_res_analysis$ID, "WholeImage_"), `[`, 2
  )

  # Remove '_dbl' suffix from ROI column
  gabrat_res_analysis$ROI <- gsub("_dbl", "", gabrat_res_analysis$ROI)

  # Remove unnecessary columns
  gabrat_res_analysis <- gabrat_res_analysis[, !colnames(gabrat_res_analysis) %in% c("X", "ID", "Sigma")

  gabrat_res_analysis
}

# Call the function to read and process GabRat results
gabrat_res_analysis <- read_gabrat(data_location, Animal_Info_ROI)

# Check the processed data
print(dim(gabrat_res_analysis)) # Should show 18 rows and 5 columns
```

```
## [1] 18  5
```

```r
print(head(gabrat_res_analysis)) # Displays first two rows of data
```

```
##             Species                            Ind Dist    GabRat               ROI
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm 0.1682120            animal
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm 0.5332068 animal+background
## 3 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm 0.4139920        background
## 4 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm 0.1624157            animal
## 5 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm 0.4468272 animal+background
## 6 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm 0.3590715        background
```

6 Combine data sets and data preparation

6.1 Merging data sets

To merge files, we will again use the `merge()` function.

6.1.1 Particle analyses

```r
# Merging two files together
Cluster_Particle_analysis <- merge(Cluster_analysis, Particle_analysis,
  by = c("Species", "Ind", "Dist", "ROI", "ClusterID")
)
```

```r
dim(Cluster_Particle_analysis) # 52 rows and 36 columns
```

```
## [1] 52 36
```

```r
Cluster_Particle_analysis[1:2, ] # Inspect data
```

```
##             Species                              Ind Dist               ROI ClusterID Pass    lw_mean
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm            animal         1    7 0.07855250 0
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal+background         1   20 0.03184927 0
##      sw_mean       sw_sd   dbl_mean      dbl_sd  Area Coverage..pc.    RNL.X     RNL.Y RNL.Saturatio
## 1 0.06569355 0.013962437 0.07415875 0.018407013  9116     100.00000 2.074853 -1.721327       2.69591
## 2 0.03404231 0.006827025 0.03058787 0.005537348 32133      71.76229 1.437122  1.917127       2.39597
##   Count Total.Area Average.Size   X.Area   Perim.     Major     Minor    Angle     Circ.  Solidity
## 1     4       2953       738.25 24.46764 204.9117  52.96716  17.83798 79.13099 0.2135263 0.5150810  8
## 2     2      32133     16066.50 45.13759 740.6488 113.31427 101.73291 73.06783 0.4666892 0.7740756 15
##    MinFeret
## 1  31.79423
## 2 119.27169
```

6.1.2 VCA,BSA,CAA data, LEIA output and GabRat analyses

Here, we demonstrate how to collate data across the VCA, BSA, CAA data, LEIA output, and GabRat output. We have excluded the particle analysis outcome from this step.

```r
# Merge the first two data frames (VCA_analysis and LEIA_Res_analysis) together
VCA_LEIA_GabRat_analysis <- merge(VCA_analysis, LEIA_Res_analysis,
  by = c("Species", "Ind", "Dist", "ROI")
)

VCA_LEIA_GabRat_analysis[1:2, ] # Inspect data
```

```
##             Species                              Ind Dist               ROI   CAA.Sc    CAA.Jc   CAA.S
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm            animal 1.000000 1.0000000      In
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal+background 1.763881 0.5879602 2.09890
##      CAA.Ht    CAA.Qt  CAA.Scpl  CAA.Qcpl      CAA.C CAA.Sc.Hrz CAA.Sc.Vrt CAA.Jc.Hrz CAA.Jc.Vrt CAA
## 1 0.0000000       NaN       Inf       NaN 0.00000000   1.000000   1.000000  1.0000000  1.0000000
## 2 0.8373184 0.7621601 0.6437981 0.7272349 0.01618755   1.763113   1.764646  0.5877043  0.5882152   2
##   CAA.Jt.Vrt CAA.Hc.Hrz CAA.Hc.Vrt CAA.Qc.Hrz CAA.Qc.Vrt CAA.Ht.Hrz CAA.Ht.Vrt CAA.Qt.Hrz CAA.Qt.Vrt
## 1        NaN    0.00000  0.0000000        NaN        NaN  0.0000000  0.0000000        NaN        NaN
## 2  0.7062285    0.76029  0.7608685  0.6920457  0.6925724  0.8386991  0.8298446  0.7634168  0.7553571
##   CAA.Qcpl.Hrz CAA.Qcpl.Vrt  CAA.C.Hrz  CAA.C.Vrt   CAA.PT CAA.PT.Hrz CAA.PT.Vrt   CAA.Asp     VCA.M
## 1          NaN          NaN 0.00000000 0.00000000      Inf        Inf        Inf       NaN 0.07415875
## 2    0.7277313    0.7239647 0.01768539 0.01469036 60.77585   55.54384   67.07187 0.4529912 0.04150538
##     VCA.sDmax VCA.CVDmax VCA.MSsat VCA.sSsat VCA.CVSsat  VCA.MSL VCA.sSL  VCA.CVSL   VCA.MS   VCA.sS
## 1         NaN        NaN  2.695919       NaN        NaN      NaN     NaN       NaN      NaN      NaN
## 2 0.009909556  0.1157695  2.469105 0.2776953   0.112468 16.01608 6.01111 0.3753171 3.334727 1.209654
##   VCA.CVL.Hrz VCA.ML.Vrt VCA.sL.Vrt VCA.CVL.Vrt VCA.MDmax.Hrz VCA.sDmax.Hrz VCA.CVDmax.Hrz VCA.MDmax
## 1         NaN 0.07415875        NaN         NaN    0.08914595           NaN            NaN    0.0891
## 2   0.5562057 0.04130759  0.0229682   0.5560287    0.08557285   0.009828942      0.1148605    0.0855
##   VCA.MSsat.Hrz VCA.sSsat.Hrz VCA.CVSsat.Hrz VCA.MSsat.Vrt VCA.sSsat.Vrt VCA.CVSsat.Vrt VCA.MSL.Hrz
## 1      2.695919           NaN            NaN      2.695919           NaN            NaN         NaN
```

```
## 2      2.467878     0.2756307      0.1116873      2.467834     0.2755308      0.1116488    16.01783
##   VCA.sSL.Vrt VCA.CVSL.Vrt VCA.MS.Hrz VCA.sS.Hrz VCA.CVS.Hrz VCA.MS.Vrt VCA.sS.Vrt VCA.CVS.Vrt   BSA
## 1         NaN          NaN        NaN        NaN         NaN        NaN        NaN         NaN 0.000
## 2    6.011548    0.3753854    3.33508   1.209564   0.3626792   3.334376   1.209744   0.3628096 0.288
##   BSA.BsDmax BSA.BCVDmax BSA.BMSsat BSA.BsSsat BSA.BCVSsat BSA.BMSL BSA.BsSL BSA.BCVSL  BSA.BMS   BS
## 1 0.00000000         NaN 0.00000000 0.00000000         NaN  0.00000 0.000000       NaN 0.000000 0.00
## 2 0.08357918    1.062552 0.06068302 0.08350352    1.376061 11.98163 4.230545 0.3530861 2.522453 0.85
##   BSA.BsL.Hrz BSA.BCVL.Hrz BSA.BML.Vrt BSA.BsL.Vrt BSA.BCVL.Vrt BSA.BMDmax.Hrz BSA.BsDmax.Hrz BSA.BC
## 1  0.00000000          NaN   0.0000000  0.00000000          NaN     0.00000000     0.00000000
## 2  0.09529334    0.3242277   0.2827497  0.09347208    0.3305825     0.07536487     0.08251804
##   BSA.BsDmax.Vrt BSA.BCVDmax.Vrt BSA.BMSsat.Hrz BSA.BsSsat.Hrz BSA.BCVSsat.Hrz BSA.BMSsat.Vrt BSA.Bs
## 1     0.00000000             NaN     0.00000000      0.0000000             NaN     0.00000000     0.
## 2     0.08458379        1.023733     0.05763893      0.0828074        1.436658     0.06434617     0.
##   BSA.BMSL.Hrz BSA.BsSL.Hrz BSA.BCVSL.Hrz BSA.BMSL.Vrt BSA.BsSL.Vrt BSA.BCVSL.Vrt BSA.BMS.Hrz BSA.Bs
## 1      0.00000      0.00000           NaN      0.00000     0.000000           NaN    0.000000   0.00
## 2     12.20617      4.27166     0.3499591     11.71142     4.156481     0.3549083    2.567851   0.86
##   BSA.BsS.Vrt BSA.BCVS.Vrt  Col.mean    Col.sd  Col.CoV Col.skew Col.kurtosis  Lum.mean   Lum.sd  Lu
## 1   0.0000000          NaN 0.1736449 0.2851837 1.642339 2.351301     9.700354 0.8928419 1.431054 1.6
## 2   0.8400862    0.3404161 0.2161124 0.4168882 1.929034 3.158602    19.114901 0.7599581 1.631294 2.1
##   Col.mean.hrz Col.sd.hrz Col.CoV.hrz Col.skew.hrz Col.kurtosis.hrz Col.mean.vrt Col.sd.vrt Col.CoV.
## 1   0.08770301  0.1856711    2.117043     3.926095         22.61682   0.08907545  0.1942661    2.180
## 2   0.11245671  0.2857060    2.540587     5.473994         47.54446   0.10647940  0.2643248    2.482
##   Lum.mean.hrz Lum.sd.hrz Lum.CoV.hrz Lum.skew.hrz Lum.kurtosis.hrz Lum.mean.vrt Lum.sd.vrt Lum.CoV.
## 1    0.4355562  0.8958863    2.056879     3.612730         20.01520    0.4563516  0.9904756    2.170
## 2    0.3860825  1.0964629    2.839970     7.414604         84.33807    0.3751880  1.0664402    2.842
##   Transform Col.thresh Lum.thresh
## 1      none          0          0
## 2      none          0          0
```

```r
dim(VCA_LEIA_GabRat_analysis) # 18 rows and 164 columns
```

```
## [1]  18 164
```

```r
# Merge the new data frame (VCA_LEIA_GabRat_analysis) with gabrat_res_analysis
VCA_LEIA_GabRat_analysis <- merge(VCA_LEIA_GabRat_analysis, gabrat_res_analysis,
  by = c("Species", "Ind", "Dist", "ROI")
)

VCA_LEIA_GabRat_analysis[1:2, ] # Inspect data again
```

```
##             Species                              Ind Dist               ROI   CAA.Sc    CAA.Jc   CAA.S
## 1 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm            animal 1.000000 1.0000000      In
## 2 aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal+background 1.763881 0.5879602 2.09890
##      CAA.Ht    CAA.Qt  CAA.Scpl  CAA.Qcpl      CAA.C CAA.Sc.Hrz CAA.Sc.Vrt CAA.Jc.Hrz CAA.Jc.Vrt CAA
## 1 0.0000000       NaN       Inf       NaN 0.00000000   1.000000   1.000000  1.0000000  1.0000000
## 2 0.8373184 0.7621601 0.6437981 0.7272349 0.01618755   1.763113   1.764646  0.5877043  0.5882152   2
##   CAA.Jt.Vrt CAA.Hc.Hrz CAA.Hc.Vrt CAA.Qc.Hrz CAA.Qc.Vrt CAA.Ht.Hrz CAA.Ht.Vrt CAA.Qt.Hrz CAA.Qt.Vrt
## 1        NaN    0.00000  0.0000000        NaN        NaN  0.0000000  0.0000000        NaN        NaN
## 2  0.7062285    0.76029  0.7608685  0.6920457  0.6925724  0.8386991  0.8298446  0.7634168  0.7553571
##   CAA.Qcpl.Hrz CAA.Qcpl.Vrt  CAA.C.Hrz  CAA.C.Vrt   CAA.PT CAA.PT.Hrz CAA.PT.Vrt   CAA.Asp     VCA.M
## 1          NaN          NaN 0.00000000 0.00000000      Inf        Inf        Inf       NaN 0.0741587
## 2    0.7277313    0.7239647 0.01768539 0.01469036 60.77585   55.54384   67.07187 0.4529912 0.0415053
##     VCA.sDmax VCA.CVDmax VCA.MSsat VCA.sSsat VCA.CVSsat  VCA.MSL VCA.sSL  VCA.CVSL   VCA.MS   VCA.sS
```

```
## 1         NaN        NaN  2.695919       NaN        NaN      NaN     NaN       NaN      NaN      NaN
## 2 0.009909556  0.1157695  2.469105 0.2776953   0.112468 16.01608 6.01111 0.3753171 3.334727 1.209654
##   VCA.CVL.Hrz VCA.ML.Vrt VCA.sL.Vrt VCA.CVL.Vrt VCA.MDmax.Hrz VCA.sDmax.Hrz VCA.CVDmax.Hrz VCA.MDmax
## 1         NaN 0.07415875        NaN         NaN    0.08914595           NaN            NaN    0.0891
## 2   0.5562057 0.04130759  0.0229682   0.5560287    0.08557285   0.009828942      0.1148605    0.0855
##   VCA.MSsat.Hrz VCA.sSsat.Hrz VCA.CVSsat.Hrz VCA.MSsat.Vrt VCA.sSsat.Vrt VCA.CVSsat.Vrt VCA.MSL.Hrz
## 1      2.695919           NaN            NaN      2.695919           NaN            NaN         NaN
## 2      2.467878     0.2756307      0.1116873      2.467834     0.2755308      0.1116488    16.01783
##   VCA.sSL.Vrt VCA.CVSL.Vrt VCA.MS.Hrz VCA.sS.Hrz VCA.CVS.Hrz VCA.MS.Vrt VCA.sS.Vrt VCA.CVS.Vrt   BSA
## 1         NaN          NaN        NaN        NaN         NaN        NaN        NaN         NaN 0.000
## 2    6.011548    0.3753854    3.33508   1.209564   0.3626792   3.334376   1.209744   0.3628096 0.288
##   BSA.BsDmax BSA.BCVDmax BSA.BMSsat BSA.BsSsat BSA.BCVSsat BSA.BMSL BSA.BsSL BSA.BCVSL  BSA.BMS   BS
## 1 0.00000000         NaN 0.00000000 0.00000000         NaN  0.00000 0.000000       NaN 0.000000 0.00
## 2 0.08357918    1.062552 0.06068302 0.08350352    1.376061 11.98163 4.230545 0.3530861 2.522453 0.85
##   BSA.BsL.Hrz BSA.BCVL.Hrz BSA.BML.Vrt BSA.BsL.Vrt BSA.BCVL.Vrt BSA.BMDmax.Hrz BSA.BsDmax.Hrz BSA.BC
## 1  0.00000000          NaN   0.0000000  0.00000000          NaN     0.00000000     0.00000000
## 2  0.09529334    0.3242277   0.2827497  0.09347208    0.3305825     0.07536487     0.08251804
##   BSA.BsDmax.Vrt BSA.BCVDmax.Vrt BSA.BMSsat.Hrz BSA.BsSsat.Hrz BSA.BCVSsat.Hrz BSA.BMSsat.Vrt BSA.Bs
## 1     0.00000000             NaN     0.00000000      0.0000000             NaN     0.00000000     0.
## 2     0.08458379        1.023733     0.05763893      0.0828074        1.436658     0.06434617     0.
##   BSA.BMSL.Hrz BSA.BsSL.Hrz BSA.BCVSL.Hrz BSA.BMSL.Vrt BSA.BsSL.Vrt BSA.BCVSL.Vrt BSA.BMS.Hrz BSA.Bs
## 1      0.00000      0.00000           NaN      0.00000     0.000000           NaN    0.000000   0.00
## 2     12.20617      4.27166     0.3499591     11.71142     4.156481     0.3549083    2.567851   0.86
##   BSA.BsS.Vrt BSA.BCVS.Vrt  Col.mean    Col.sd  Col.CoV Col.skew Col.kurtosis  Lum.mean   Lum.sd  Lu
## 1   0.0000000          NaN 0.1736449 0.2851837 1.642339 2.351301     9.700354 0.8928419 1.431054 1.6
## 2   0.8400862    0.3404161 0.2161124 0.4168882 1.929034 3.158602    19.114901 0.7599581 1.631294 2.1
##   Col.mean.hrz Col.sd.hrz Col.CoV.hrz Col.skew.hrz Col.kurtosis.hrz Col.mean.vrt Col.sd.vrt Col.CoV.
## 1   0.08770301  0.1856711    2.117043     3.926095         22.61682   0.08907545  0.1942661    2.180
## 2   0.11245671  0.2857060    2.540587     5.473994         47.54446   0.10647940  0.2643248    2.482
##   Lum.mean.hrz Lum.sd.hrz Lum.CoV.hrz Lum.skew.hrz Lum.kurtosis.hrz Lum.mean.vrt Lum.sd.vrt Lum.CoV.
## 1    0.4355562  0.8958863    2.056879     3.612730         20.01520    0.4563516  0.9904756    2.170
## 2    0.3860825  1.0964629    2.839970     7.414604         84.33807    0.3751880  1.0664402    2.842
##   Transform Col.thresh Lum.thresh    GabRat
## 1      none          0          0 0.1682120
## 2      none          0          0 0.5332068
```

```r
dim(VCA_LEIA_GabRat_analysis) # 18 rows and 165 columns
```

```
## [1]  18 165
```

6.2 Data cleaning

In this tutorial, we are using a minimal working example, where the data used does not have proper replication (i.e., we have 3 individuals in total). For those considering using linear mixed effects models (LMMs) in R for downstream analyses, we recommend reading Harrison et al. (2018). Please consider the number of data points (n) to parameters (k); where ideally $n/k > 3$ (but see discussion in Harrison et al. 2018).

6.2.1 Omit rows based on values

To demonstrate some data cleaning methods, we have created a smaller data frame `V.L.GbRt_sub_analysis`, subset from `VCA_LEIA_GabRat_analysis` by selecting the columns names. In this data set, there are both `NaN` (not a number) and `Inf` (infinite) entries. Using `summary()` we can see that by retaining `NaN` and `Infs`,

functions like `mean()` and `max()` can not calculated values correctly. To remove `Inf` values, we first convert them to `NA`. To do so, we can use the `is.infinite()` function to detect `Inf` values, however this only works on vectors (i.e., data frame columns), so we can loop over our data frame using a variant of `lapply()`, the `sapply()` function which returns a vector (not list).

```
# Specify column names
ColumnNames <- c(
  "Species", "Ind", "Dist", "ROI", "BSA.BsL.Hrz", "Lum.mean",
  "CAA.Scpl", "Lum.CoV", "BSA.BML", "Col.mean", "GabRat"
)

# Subset data by column names
V.L.GbRt_sub_analysis <- VCA_LEIA_GabRat_analysis[, colnames(VCA_LEIA_GabRat_analysis)
%in% ColumnNames]

dim(V.L.GbRt_sub_analysis) # 18 rows by 11 columns
```

```
## [1] 18 11
```

```
summary(V.L.GbRt_sub_analysis) # Initial summary of the data
```

```
##               Species                                Ind      Dist                  ROI       CAA.Scpl
##  aphelodoris_varia:12   3_AphelodorisVaria_10_NF_D_RAW:6   30cm:9   animal           :6   Min.   :0.3
##  aplysia_sp       : 6   3_AphelodorisVaria_11_NF_D_RAW:6   50cm:9   animal+background:6   1st Qu.:0.5
##                         4_Aplysiasp_1_NL_N_RAW        :6            background       :6   Median :0.6
##                                                                                          Mean   :
##                                                                                          3rd Qu.:0.8
##                                                                                          Max.   :
##
##   BSA.BsL.Hrz         Col.mean         Lum.mean         Lum.CoV          GabRat
##  Min.   :0.00000   Min.   :0.1542   Min.   :0.5980   Min.   :1.483   Min.   :0.1624
##  1st Qu.:0.04736   1st Qu.:0.1827   1st Qu.:0.7779   1st Qu.:1.709   1st Qu.:0.2622
##  Median :0.07755   Median :0.1937   Median :0.8958   Median :1.864   Median :0.3781
##  Mean   :0.07439   Mean   :0.2068   Mean   :0.9352   Mean   :1.904   Mean   :0.3521
##  3rd Qu.:0.10934   3rd Qu.:0.2291   3rd Qu.:1.0017   3rd Qu.:2.062   3rd Qu.:0.4386
##  Max.   :0.15655   Max.   :0.2857   Max.   :1.5512   Max.   :2.450   Max.   :0.5332
##  NA's   :4
```

```
# View the data frame
View(V.L.GbRt_sub_analysis)

# Replace infinite values with NA using sapply
V.L.GbRt_sub_analysis[sapply(V.L.GbRt_sub_analysis, is.infinite)] <- NA

View(V.L.GbRt_sub_analysis) # Observe change the data frame

# Remove NA/NaNs
V.L.GbRt_sub_analysis <- na.omit(V.L.GbRt_sub_analysis)

View(V.L.GbRt_sub_analysis) # Observe change the data frame

dim(V.L.GbRt_sub_analysis) # 11 rows by 11 columns
```

```
## [1] 11 11
```

```r
summary(V.L.GbRt_sub_analysis) # Inspect column distributions
```

```
##                Species                                Ind      Dist                   ROI       CAA.Scpl
##  aphelodoris_varia:6   3_AphelodorisVaria_10_NF_D_RAW:2   30cm:7   animal           :3   Min.   :0.30
##  aplysia_sp       :5   3_AphelodorisVaria_11_NF_D_RAW:4   50cm:4   animal+background:5   1st Qu.:0.47
##                        4_Aplysiasp_1_NL_N_RAW        :5            background       :3   Median :0.53
##                                                                                          Mean   :0.52
##                                                                                          3rd Qu.:0.59
##                                                                                          Max.   :0.68
##   BSA.BsL.Hrz         Col.mean         Lum.mean         Lum.CoV          GabRat
##  Min.   :0.04544   Min.   :0.1542   Min.   :0.5980   Min.   :1.483   Min.   :0.1906
##  1st Qu.:0.06180   1st Qu.:0.1881   1st Qu.:0.8488   1st Qu.:1.769   1st Qu.:0.2678
##  Median :0.09529   Median :0.2161   Median :0.9803   Median :2.040   Median :0.4105
##  Mean   :0.09468   Mean   :0.2179   Mean   :1.0176   Mean   :1.983   Mean   :0.3775
##  3rd Qu.:0.11521   3rd Qu.:0.2534   3rd Qu.:1.0842   3rd Qu.:2.104   3rd Qu.:0.4647
##  Max.   :0.15655   Max.   :0.2857   Max.   :1.5512   Max.   :2.450   Max.   :0.5332
```

```r
# Inspect the variance for each colour variable
apply(V.L.GbRt_sub_analysis[, 5:11], 2, var)
```

```
##    CAA.Scpl     BSA.BML BSA.BsL.Hrz    Col.mean    Lum.mean     Lum.CoV      GabRat
## 0.012682464 0.006015432 0.001430767 0.002207957 0.087193128 0.090558647 0.015336849
```

6.2.2 Testing for normality and univaraite outliers

Again, with our minimal example, our estimates do not deviate from normality but here we have provided code to test this. Using a combination of the Shapiro-Wilk test for normality (Part A) and histograms (Part B) we can determine if our data is normal. For the Shapiro-Wilk test, the null hypothesis is that data is normal (i.e. p-value less than 0.05 are not normally distributed).

```r
# Part A:
# Test for normality Shapiro-Wilk test
lapply(V.L.GbRt_sub_analysis[, 5:11], shapiro.test)
```

```
## $CAA.Scpl
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.95222, p-value = 0.6724
##
##
## $BSA.BML
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.97579, p-value = 0.9383
##
```

```
##
## $BSA.BsL.Hrz
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.94014, p-value = 0.5222
##
##
## $Col.mean
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.92552, p-value = 0.3674
##
##
## $Lum.mean
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.90032, p-value = 0.1865
##
##
## $Lum.CoV
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.95266, p-value = 0.6781
##
##
## $GabRat
##
##  Shapiro-Wilk normality test
##
## data:  X[[i]]
## W = 0.90139, p-value = 0.1922
```

```r
# Part B:
# Visualise distribution of the data using histograms
Colnames_subset <- colnames(V.L.GbRt_sub_analysis[, 5:11]) # Select only the colour variables

# Set the plot dimensions to display plots as 3 rows by 3 columns
par(mfrow = c(3, 3))

# Use a for loop to iterate over the column names
for (i in Colnames_subset) {
  # Plot the histogram, using 10 bins, and the colour variable as the main title
  hist(V.L.GbRt_sub_analysis[, i],
    breaks = 10,
    main = i, xlab = ""
  )
```

```
}
```

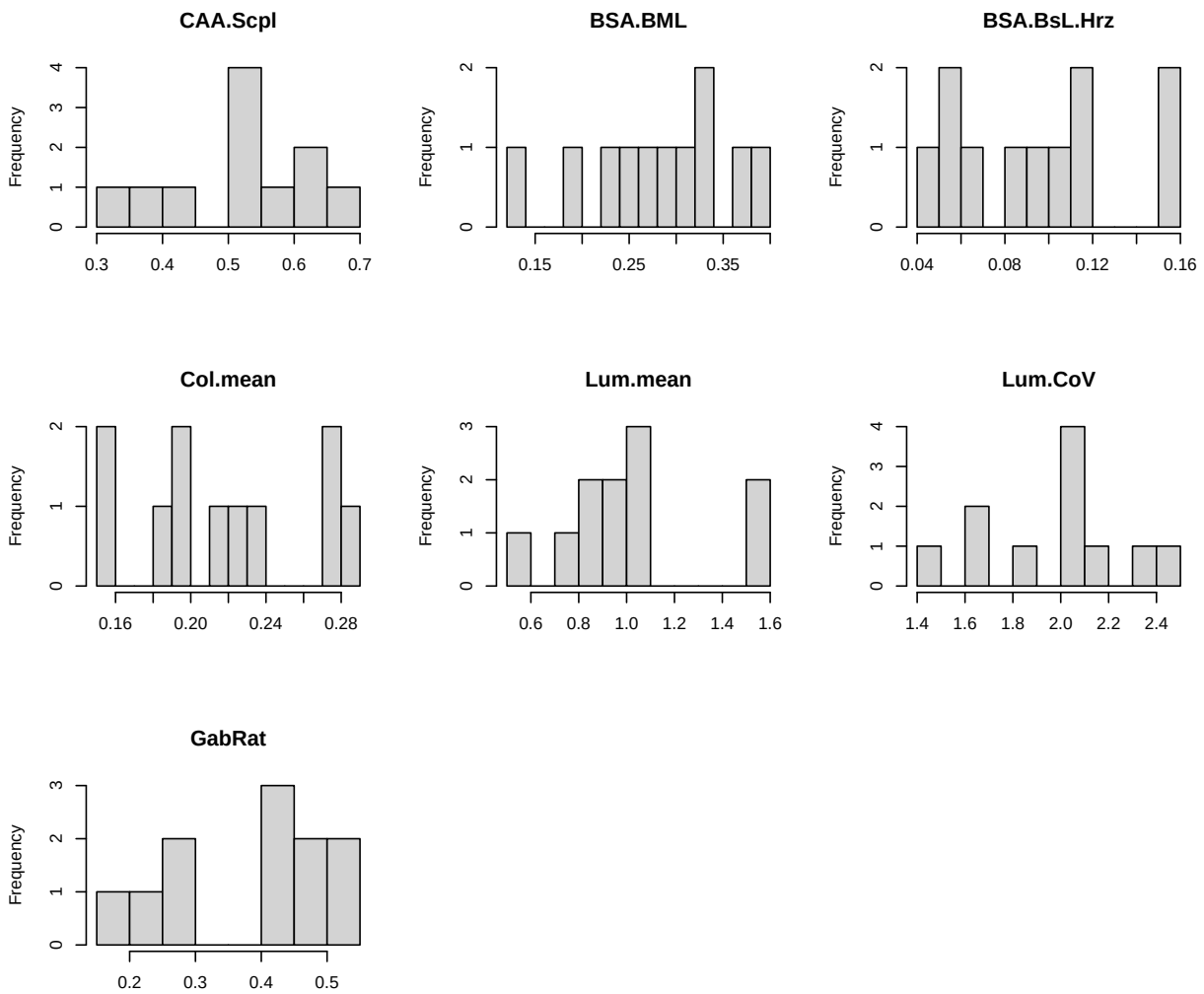

Figure 2: Figure 2. Histograms of seven colour variables.

To determine the degree of association between our colour variables, we can use both correlations and scatter plots. As our data is normally distributed we can use Pearson's correlation.

```
# Correlation panel
panel.cor <- function(x, y) {
  usr <- par("usr")
  on.exit(par(usr))
  par(usr = c(0, 1, 0, 1))

  # Calculate Pearson's correlation
  r <- round(cor(x, y, method = "pearson"), digits = 2)

  # Save as text
  txt <- paste0("R = ", r)
```

```r
  # Plot the text
  text(0.5, 0.5, txt, cex = 1)
}

my_cols <- c("#00AFBB", "#E7B800", "#FC4E07")

# Scatter plot panel
panel.scat <- function(x, y) {
  # Specify scatter plot points to colour by ROI
  points(x, y,
    col = my_cols[V.L.GbRt_sub_analysis$ROI],
    cex = 1.1, pch = 19
  )
}

pairs(V.L.GbRt_sub_analysis[, 5:11],
  # Set title
  main = "Seven colour variables for two viewing distances",

  # Parse in the new panels
  upper.panel = panel.cor,
  lower.panel = panel.scat,

  # Add margin space and reduce gaps between plots
  oma = c(5, 5, 6, 18), gap = 0
)

# Use the data to create a figure legend
Colour_ID <- unique(data.frame(
  as.character(V.L.GbRt_sub_analysis$ROI),
  my_cols[V.L.GbRt_sub_analysis$ROI]
))

# Add figure legend to plot
legend("right", Colour_ID[, 1],
  xpd = TRUE,
  pch = 19, cex = 1.1, bty = "n", col = Colour_ID[, 2]
)
```

From the scatter plots in Figure 3, there doesn't appear to be any outliers in each of the colour variables, however we can confirm this using a boxplot (Part A) and then calculate the z-scores for each estimate (Part B) and look for outliers at greater than 3 standard deviations.

```r
# Part A:
# For the boxplot function, the data has to be in long format
# We need the reshape package for this
library(reshape2)

# Here we 'melt' the data to long format
V.L.GbRt_sub_analysis_lng <- melt(V.L.GbRt_sub_analysis,
  id.vars = c("Species", "Ind", "Dist", "ROI")
)
```

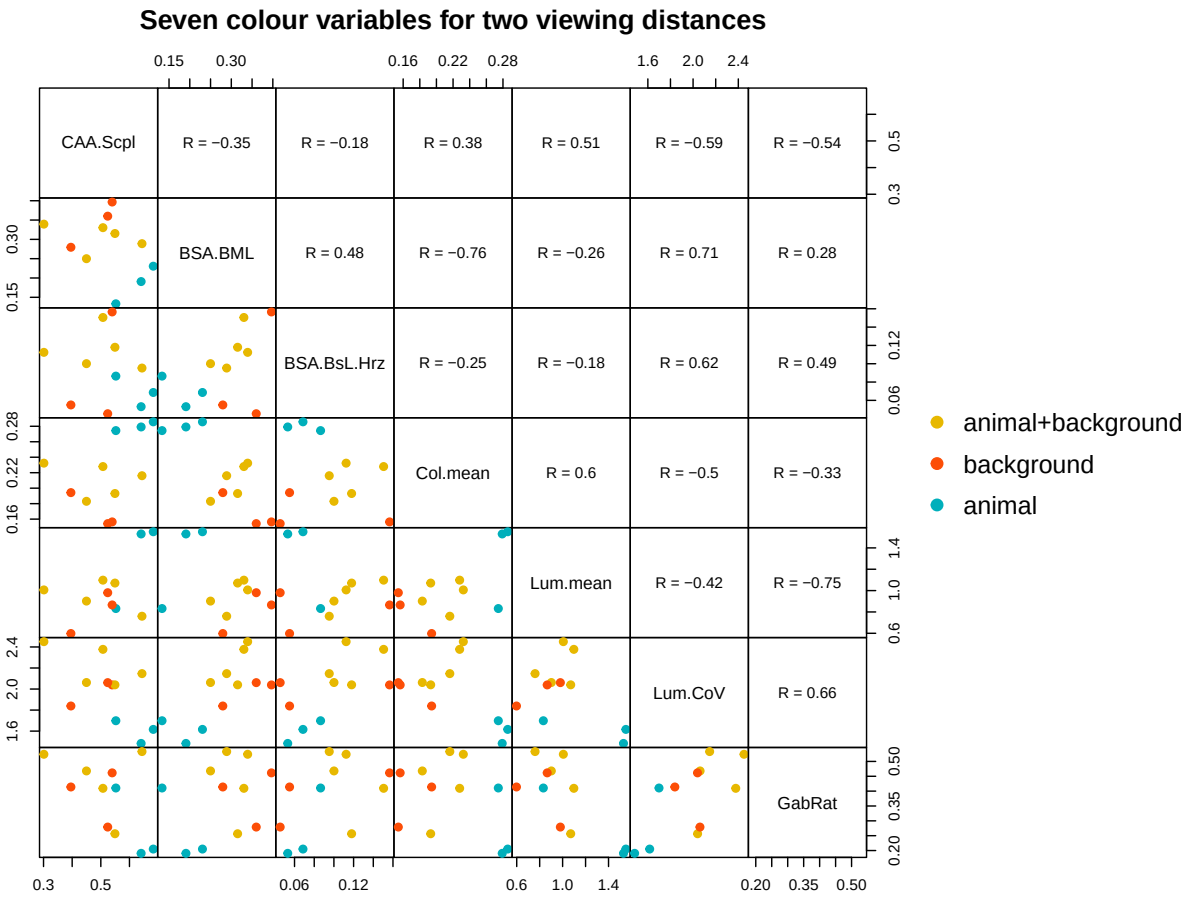

Figure 3: Figure 3. Paired scatter plots of seven colour variables. Lower off-diagonal are the paired scatterplots where coloured points represent regions of interest (animal+background is yellow, bacground is red and animal is blue). Upper off-diagonal represents Pearson's correlation between paired colur varaibles.

```
## Warning in melt(V.L.GbRt_sub_analysis, id.vars = c("Species", "Ind", "Dist", : The melt generic in da
## data.frame and will attempt to redirect to the relevant reshape2 method; please note that reshape2 is
## is now deprecated as well. To continue using melt methods from reshape2 while both libraries are atta
## prepend the namespace like reshape2::melt(V.L.GbRt_sub_analysis). In the next version, this warning w
```

```
# Set the plot dimensions to display a single plot, correcting outer margins
par(mfrow = c(1, 1), oma = c(0, 0, 0, 0))

# Inspect boxplots for outliers
boxplot(value ~ variable,
  pch = 16, boxwex = 0.5, xlab = "Colour variable",
  data = V.L.GbRt_sub_analysis_lng
)
```

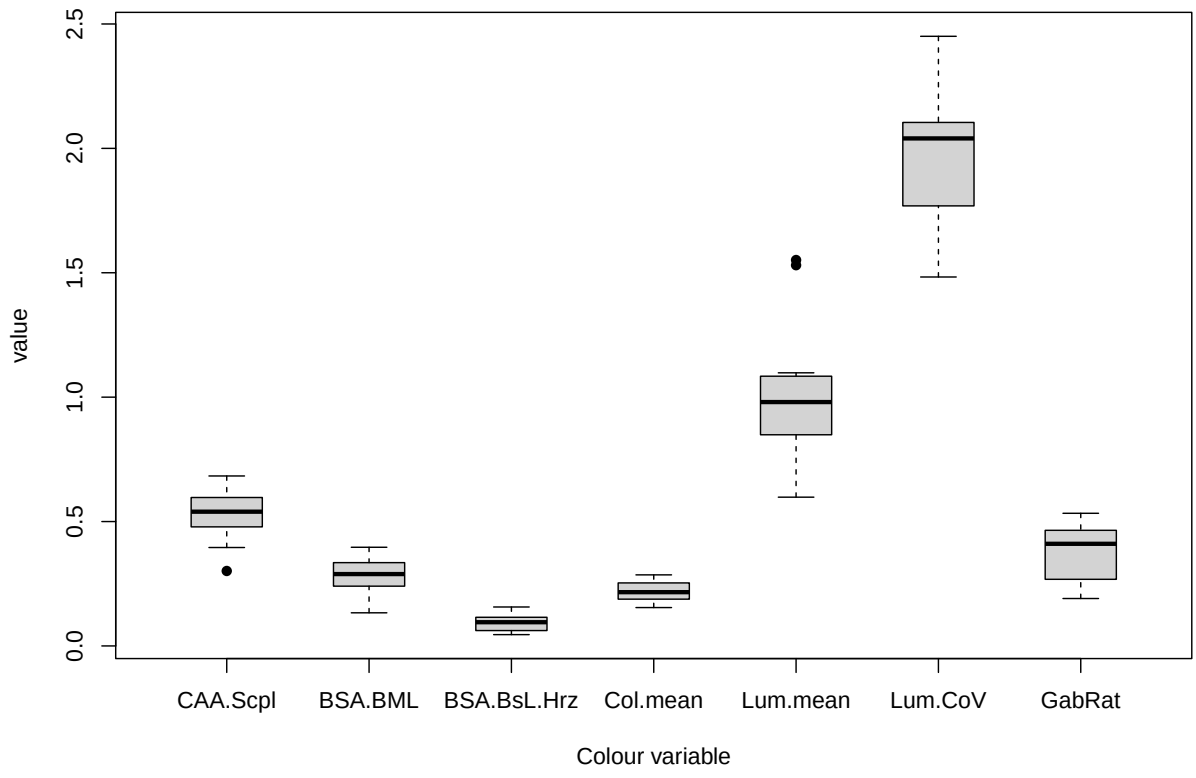

Figure 4: Figure 4. Boxplots of the seven colour variables.

```
# Calculate z-scores and look for outliers
apply(V.L.GbRt_sub_analysis[, 5:11], 2, function(x) (x - mean(x)) / sd(x))
```

```
##      CAA.Scpl     BSA.BML BSA.BsL.Hrz    Col.mean    Lum.mean    Lum.CoV     GabRat
## 2   1.0447329  0.07370189  0.01615151 -0.03748558 -0.87259277  0.5434126  1.2575825
## 3  -1.1591094 -0.04606569 -1.04912193 -0.50269435 -1.42120580 -0.4793016  0.2949464
## 7   1.3944807 -0.67822953 -0.68927858  1.44389073  1.80698261 -1.2187555 -1.3922980
```

```
## 8  -1.9965592  0.72545366  0.46902751  0.30774049 -0.03805163  1.5533929  1.1835279
## 10  1.0175188 -1.19110556 -1.09924489  1.30323751  1.73650453 -1.6615115 -1.5085298
## 11 -0.1664538  0.60802358  1.47594374  0.21417235  0.27164927  1.3104953  0.2588835
## 13  0.2343956 -1.93327402 -0.21669972  1.20297119 -0.62888125 -0.9445464  0.2669827
## 14 -0.6773729 -0.42764791  0.14299748 -0.74177818 -0.39271849  0.2623784  0.7306342
## 15  0.1198638  1.46361125  1.63571008 -1.30800444 -0.51469509  0.1861873  0.6787438
## 17  0.2065297  0.41346282  0.61625213 -0.52704120  0.17954864  0.1893369 -0.9760417
## 18 -0.0180262  0.99206954 -1.30173733 -1.35500854 -0.12654002  0.2589115 -0.7944315
```

Although in Figure 4, there are two points in Lum.mean that exceed 1.5 times the interquartile range (the outer whiskers of the boxplot), none of the points exceed 3 standard deviations.

6.2.3 Standardising data

When implementing multivariate models, it is best to standardize data to facilitate model convergence. Here we variance standardised (mean = 0, standard deviation = 1) each colour pattern variable across both viewing distances and ROIs using the `scale()` function.

```r
# Inspect scaled values and column means
scale(V.L.GbRt_sub_analysis[, 5:11], center = TRUE, scale = TRUE)
```

```
##      CAA.Scpl     BSA.BML BSA.BsL.Hrz    Col.mean    Lum.mean    Lum.CoV     GabRat
## 2   1.0447329  0.07370189  0.01615151 -0.03748558 -0.87259277  0.5434126  1.2575825
## 3  -1.1591094 -0.04606569 -1.04912193 -0.50269435 -1.42120580 -0.4793016  0.2949464
## 7   1.3944807 -0.67822953 -0.68927858  1.44389073  1.80698261 -1.2187555 -1.3922980
## 8  -1.9965592  0.72545366  0.46902751  0.30774049 -0.03805163  1.5533929  1.1835279
## 10  1.0175188 -1.19110556 -1.09924489  1.30323751  1.73650453 -1.6615115 -1.5085298
## 11 -0.1664538  0.60802358  1.47594374  0.21417235  0.27164927  1.3104953  0.2588835
## 13  0.2343956 -1.93327402 -0.21669972  1.20297119 -0.62888125 -0.9445464  0.2669827
## 14 -0.6773729 -0.42764791  0.14299748 -0.74177818 -0.39271849  0.2623784  0.7306342
## 15  0.1198638  1.46361125  1.63571008 -1.30800444 -0.51469509  0.1861873  0.6787438
## 17  0.2065297  0.41346282  0.61625213 -0.52704120  0.17954864  0.1893369 -0.9760417
## 18 -0.0180262  0.99206954 -1.30173733 -1.35500854 -0.12654002  0.2589115 -0.7944315
## attr(,"scaled:center")
##    CAA.Scpl     BSA.BML BSA.BsL.Hrz    Col.mean    Lum.mean     Lum.CoV      GabRat
##   0.5261440   0.2831279   0.0946824   0.2178738   1.0176215   1.9830285   0.3774653
## attr(,"scaled:scale")
##    CAA.Scpl     BSA.BML BSA.BsL.Hrz    Col.mean    Lum.mean     Lum.CoV      GabRat
##  0.11261644  0.07755922  0.03782548  0.04698890  0.29528483  0.30092964  0.12384203
```

```r
# Update the V.L.GbRt_sub_analysis data frame with the standardised values
V.L.GbRt_sub_analysis[, 5:11] <- scale(V.L.GbRt_sub_analysis[, 5:11],
  center = TRUE, scale = TRUE
)

# Check standardisation
summary(V.L.GbRt_sub_analysis) # Mean values are zero
```

```
##               Species                                Ind      Dist                   ROI       CAA.Scpl
##  aphelodoris_varia:6   3_AphelodorisVaria_10_NF_D_RAW:2   30cm:7   animal           :3   Min.   :-1.9
##  aplysia_sp       :5   3_AphelodorisVaria_11_NF_D_RAW:4   50cm:4   animal+background:5   1st Qu.:-0.4
##                        4_Aplysiasp_1_NL_N_RAW        :5            background       :3   Median : 0.1
```

```
##                                                                                                                    Mean   : 0.0
##                                                                                                                    3rd Qu.: 0.0
##                                                                                                                    Max.   : 1.3
##   BSA.BsL.Hrz          Col.mean           Lum.mean          Lum.CoV            GabRat
##  Min.   :-1.30174   Min.   :-1.35501   Min.   :-1.4212   Min.   :-1.6615   Min.   :-1.5085
##  1st Qu.:-0.86920   1st Qu.:-0.63441   1st Qu.:-0.5718   1st Qu.:-0.7119   1st Qu.:-0.8852
##  Median : 0.01615   Median :-0.03749   Median :-0.1265   Median : 0.1893   Median : 0.2670
##  Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.: 0.54264   3rd Qu.: 0.75536   3rd Qu.: 0.2256   3rd Qu.: 0.4029   3rd Qu.: 0.7047
##  Max.   : 1.63571   Max.   : 1.44389   Max.   : 1.8070   Max.   : 1.5534   Max.   : 1.2576
```

```r
apply(V.L.GbRt_sub_analysis[, 5:11], 2, var) # Confirm variances are 1
```

```
##    CAA.Scpl     BSA.BML BSA.BsL.Hrz    Col.mean    Lum.mean     Lum.CoV      GabRat
##           1           1           1           1           1           1           1
```

6.2.4 Multivaraite outliers

For users who will be investigating colours multivariately, we would recommend using Mahalanobis distance to determine if there are any outliers in multivariate space. To determine p-values for Mahalanobis distance, you can use the Chi-Square statistic with k-1 degrees of freedom, where k is the number of variables (here k = 7), and p-values less than 0.05 outliers.

```r
# Using R's Mahalanobis function, add a new column to your data
V.L.GbRt_sub_analysis$mahalnobis <- mahalanobis(
  V.L.GbRt_sub_analysis[, 5:11],

  # Get a vector of column means
  colMeans(V.L.GbRt_sub_analysis[, 5:11]),

  # Get a covaraince matrix
  cov(V.L.GbRt_sub_analysis[, 5:11])
)

# Calcualte p-values using k-1 df = 6
V.L.GbRt_sub_analysis$pvalue <- pchisq(V.L.GbRt_sub_analysis$mahalnobis,
  df = 6, lower.tail = FALSE
)

# Inspect Mahalnobis distances and p-values
V.L.GbRt_sub_analysis[, colnames(V.L.GbRt_sub_analysis) %in% c("mahalnobis", "pvalue")]
```

```
##    mahalnobis    pvalue
## 2    7.941035 0.2424552
## 3    6.662854 0.3531541
## 7    4.371090 0.6265923
## 8    7.589055 0.2697817
## 10   4.524537 0.6060684
## 11   4.608575 0.5949021
## 13   6.112840 0.4106692
## 14   8.005528 0.2376986
## 15   8.490756 0.2043073
## 17   5.134891 0.5266326
## 18   6.558839 0.3635764
```

7 Export files

Depending on what image analysis sets you are using, we have provided multiple ways to save the image analysis data files as csvs in the folder 'Output'. First, we use `dir.create()` to create the folder 'Output' in the `test_data` folder, and save this new folder location as `Out_path`. To save a single file, you can use `write.csv()`, where we have used the argument `row.names = FALSE` to make sure that the row numbers in the dataframe aren't included in the csv file.

To save mutliple files, we utilise the fact that our naming convention in this R tutorial specifies image analysis data files with `_analysis`. Using the `ls()` function with the pattern argument set to `_analysis`, we can create a list of all our image analysis dataframes. Then, using a for loop, we iterate over each dataframe, rename it using the current date, and then output the csv file. Finally, we use `list.files()` to check that the files have been created.

```
dir.create(paste(c(data_location, "Output"), collapse = "/")) # Create the Output directory

Out_path <- paste(c(data_location, "Output"), collapse = "/") # Save Output directory path

# For a single file:
write.csv(gabrat_res_analysis,
  row.names = FALSE, # Remove row names
  file = paste(c(Out_path, "gabrat_res_analysis.csv"),
    collapse = "/"
  )
) # Save to the folder Output in test_data

# For multiple files:
savefile_list <- ls(pattern = "analysis") # Save all objects with "analysis" in the name

savefile_list # Inspect object names
```

```
##  [1] "Cluster_analysis"          "Cluster_Particle_analysis" "gabrat_res_analysis"       "IndParticle
##  [5] "LEIA_Res_analysis"         "Particle_analysis"         "V.L.GbRt_sub_analysis"     "V.L.GbRt_su
##  [9] "VCA_analysis"              "VCA_LEIA_GabRat_analysis"
```

```
# Iterate over each data frame in savefile_list
for (i in savefile_list) {
  # Create file name and add the .csv extension
  # Add date of creation using format(Sys.time(), "_%d_%b_%Y")
  # Remove "_analysis" using gsub and add "_data" and date created
  fname <- paste0(gsub("_analysis", "", i), "_data", format(Sys.time(), "_%d_%b_%Y"), ".csv")

  # Save to the folder Output in test_data
  write.csv(gabrat_res_analysis,
    row.names = FALSE, # Remove row names
    file = paste(c(Out_path, fname), collapse = "/")
  )
}

# Check that your files have saved
list.files(Out_path)
```

```
##  [1] "Cluster_data_27_Oct_2023.csv"          "Cluster_Particle_data_27_Oct_2023.csv" "gabrat_res_anal
```

```
##  [4] "gabrat_res_data_27_Oct_2023.csv"       "IndParticle_data_27_Oct_2023.csv"      "LEIA_Res_data_2
##  [7] "Particle_data_27_Oct_2023.csv"         "V.L.GbRt_sub_data_27_Oct_2023.csv"     "V.L.GbRt_sub_lr
## [10] "VCA_data_27_Oct_2023.csv"              "VCA_LEIA_GabRat_data_27_Oct_2023.csv"
```

8 Fast import for large data sets using data.table (optional - advanced)

For very large data sets we would recommend importing data using the `data.table` package (Dowle and Srinivasan 2023). The `data.table` format can be used in parallel and is optimised for fast `.csv` reading/writing. For further information and cheatsheets, visit the `data.table` GitHub page (https://github.com/Rdatatable/data.table). The *data.table* has the following structure:

```
FROM[WHERE, SELECT, GROUP BY]
DT  [i,     j,      by]
```

Here we will provide a quick demonstration by importing the LEIA QCPA batch script output. First, we convert our `Animal_Info_ROI` into `data.table` format using the function `setDT`. Then we will input our data using the dedicated `data.table` read function `fread`. Finally, we can output large data.tables fast using the `fwrite` function.

```r
library(data.table)

# Convert Animal_Info_ROI into data.table format
Animal_Info_ROI_DT <- setDT(Animal_Info_ROI)

# From Animal_Info_ROI we can paste the column information together
# and then read in data row by row, specifying 'by=1:NROW(Animal_Info_ROI)'
LEIA_analysis_DT <- Animal_Info_ROI[, fread(paste(data_location, Species, Ind, Dist, "LEIA", ROI,
  "_Local Edge Intensity Analysis.csv",
  sep = "/"
)),
by = 1:NROW(Animal_Info_ROI)
]

# Remove unwanted columns, and automatically assign using ':='
LEIA_analysis_DT[, c("NROW", "V1", "Image", "Transform") := NULL]

# Add the Animal_Info_ROI information
LEIA_analysis_DT <- cbind(Animal_Info_ROI, LEIA_analysis_DT)

# Inspect the data.table
head(LEIA_analysis_DT)
```

```
##              Species                            Ind Dist    ROI  Col mean    Col sd  Col CoV  Col ske
## 1: aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 30cm animal 0.1736449 0.2851837 1.642339 2.35130
## 2: aphelodoris_varia 3_AphelodorisVaria_10_NF_D_RAW 50cm animal 0.1663600 0.2695337 1.620183 2.28632
## 3: aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 30cm animal 0.2857206 0.4620673 1.617200 2.11097
## 4: aphelodoris_varia 3_AphelodorisVaria_11_NF_D_RAW 50cm animal 0.2791115 0.4172110 1.494782 1.66957
## 5:        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 30cm animal 0.2744001 0.5186319 1.890057 0.98636
## 6:        aplysia_sp         4_Aplysiasp_1_NL_N_RAW 50cm animal 0.1879060 0.3406233 1.812732 0.89513
##     Lum CoV  Lum skew Lum kurtosis Col mean hrz Col sd hrz Col CoV hrz Col skew hrz Col kurtosis hrz
```

```
## 1: 1.602808 2.2559735     9.558368   0.08770301  0.1856711    2.117043     3.926095         22.61682
## 2: 1.536144 2.0603133     8.282279   0.08205933  0.1756506    2.140532     3.887552         22.67175
## 3: 1.616269 2.5463566    12.453982   0.15267951  0.3221450    2.109943     3.683355         20.80604
## 4: 1.483030 2.0211324     8.743586   0.14939446  0.2985774    1.998584     3.244117         16.06562
## 5: 1.698786 0.4658983     2.619744   0.13644481  0.3012997    2.208217     2.138189         11.13582
## 6: 1.741279 0.6844910     3.147853   0.09580569  0.1994786    2.082117     2.279845         13.36001
##    Col skew vrt Col kurtosis vrt Lum mean hrz Lum sd hrz Lum CoV hrz Lum skew hrz Lum kurtosis hrz Lu
## 1:     3.913759         22.07321    0.4355562  0.8958863    2.056879     3.612730        20.015200
## 2:     3.881844         22.90526    0.4406635  0.8761384    1.988225     3.493638        19.455513
## 3:     3.515951         18.79095    0.8198832  1.7714397    2.160600     4.465145        31.933329
## 4:     2.907237         14.14649    0.8071571  1.6670532    2.065339     3.871669        23.224363
## 5:     2.280570         10.90464    0.4030806  0.7863049    1.950739     1.070465         4.311739
## 6:     2.295968         11.87647    0.4673162  0.9458438    2.023991     1.530686         7.331437
##    Lum skew vrt Lum kurtosis vrt Col thresh Lum thresh
## 1:     3.852029         22.42973          0          0
## 2:     3.690217         20.01364          0          0
## 3:     3.836817         23.39465          0          0
## 4:     3.191745         18.53788          0          0
## 5:     1.771279         10.55201          0          0
## 6:     2.801707         15.67831          0          0
```

```r
# Create save filename
fname <- paste0(
  Out_path, "/", gsub("_analysis", "", "LEIA_analysis_DT"),
  "_data", format(Sys.time(), "_%d_%b_%Y"), ".csv"
)
# Save output
fwrite(LEIA_analysis_DT, file = fname, row.names = FALSE)
```

References

Dowle, Matt, and Arun Srinivasan. 2023. *Data.table: Extension of 'Data.frame'*. https://CRAN.R-project.org/package=data.table.

Harrison, Xavier A, Lynda Donaldson, Maria Eugenia Correa-Cano, Julian Evans, David N Fisher, Cecily ED Goodwin, Beth S Robinson, David J Hodgson, and Richard Inger. 2018. "A Brief Introduction to Mixed Effects Modelling and Multi-Model Inference in Ecology." *PeerJ* 6: e4794.

van den Berg, Cedric P, Jolyon Troscianko, John A Endler, N Justin Marshall, and Karen L Cheney. 2020. "Quantitative Colour Pattern Analysis (QCPA): A Comprehensive Framework for the Analysis of Colour Patterns in Nature." *Methods in Ecology and Evolution* 11 (2): 316–32.